Recurrence: what looping a shared block does to a path-integrated model

Expansion note for `docs/paper/MAIN_DRAFT1.md` Secs. 5, 6.1 and 6.3 (2026-10-20). Labels: **ESTABLISHED** (registered, results file named; a CORRECTED / AUDIT block wins), **ANALYSIS** (computed here on stored checkpoints, CPU, inference only unless stated; script named; all post hoc), **THEORY** (statement, assumptions, the check that could break it), **SPECULATION**. Scripts and outputs: `docs/paper/expansion/scripts/recurrence/` (`out/` holds every JSON and text table quoted below). No GPU used, nothing committed, no existing file edited.

---

0. Summary

Key insight. In MapFormer the phase theta is computed once, from the token embeddings, before any block runs (`model.py` `MapFormerWM.forward`, `model_looped.py` `MapFormerWM_Looped.forward`). A loop or extra layers therefore cannot change the position code's class: rank-2 plain (A), rank-2 looped x4 (L) and rank-2 at four real layers (L4) draw their code from exactly the same family (`token_emb`, `W_in`, `W_out`, `omega`). What extra passes change is the **reader**: in one layer the query at an action token depends only on the action's identity and the key only on the token's identity, so position reaches the score only through theta differences. From the second pass on, queries and keys carry context, and the model can read the same code in ways one pass cannot. The stored checkpoints show this is what happens (ANALYSIS):

1. **The loop and depth do not find a better code.** A one-pass, position-only reader fitted to each model's own theta (an idealised single head) scores 0.893 (L) and 0.897 (L4) against 0.866 for A (perm p 0.28 / 0.27) and 1.000 for rank 4 (p 0.0002). On long-gap revisits the code supports 0.61 / 0.65 / 0.56 / 1.00, on wrap-only revisits 0.41 / 0.45 / 0.45 / 1.00.
2. **They read it far better.** The models beat their own code's one-pass reader by +0.32 / +0.34 on long-gap and +0.34 / +0.44 on wrap-only revisits. For the one-pass models the margin is +0.11 / +0.11 (A) and -0.005 / -0.02 (rank 4) (gap vs A's gap: p 0.0025 / 0.0045 long, 0.0002 / 0.0002 wrap). A one-layer model is almost exactly its code (A +0.028 overall). The looped model is a 4-step computation, not an improved 1-layer model: run with one pass, the same weights score 0.381, below the 0.51 always-blank floor. Retrieval of the right cell appears only at passes 2-4: same-cell attention mass rises 0.03 -> 0.36 -> 0.46 -> 0.52 (head 0) while attention entropy falls 6.1 -> 2.4 nats.
3. **Rank 4's code is never found.** Classifying each step map by its omega-weighted geometry: a two-dimensional, cancelling ("2D map") code in 1/8 A, 0/8 L, 1/8 L4 and 8/8 rank-4 runs (aids pooled vs rank 4: Fisher p 1.2e-5). What the aids do reach, in 8/16 runs against 0/8 for A (Fisher p 0.022, post hoc), is a **cancelling but one-dimensional** code: the x and y phase steps are collinear (sigma_2/sigma_1 <= 0.002), so the code reads a single coordinate, x + kappa y (|kappa| 0.62-1.72; 1.38-1.42 in all four L runs). That is a clock-free code with a lost map dimension, a third end state beside the "drifting" (clock-coupled) code of failing rank-2 runs and the 2D code of rank 4.

How this strengthens the paper. The current text says rank 2 fails "by search, not capacity" (Sec. 6.1). The loop and depth results, read this way, sharpen that claim instead of muddying it. Four times the computation (L, same parameters and initialisation as A) or four times the parameters and computation (L4) lift accuracy a long way (registered: +0.079 and +0.096, perm p 0.0034 / 0.0012). The gain comes from computation downstream of the code, and the code itself stays a rank-2 failure. So the deficit is local to the content-to-angle map: nothing downstream repairs it, and spending rank is the only intervention that finds the 2D code. It also gives Sec. 5's exchange rate a second axis. Extra passes add accuracy exactly where the code carries information one pass cannot read: index code, short gaps only (+0.17; long and wrap unchanged); rank-2 path code, long gaps and wrap (+0.26-0.33, +0.19-0.33); rank-4 code, nothing left to add.

What to add. (a) One paragraph in Sec. 6.1 after "The rank-2 solution exists and is held" (text in Sec. 6.1 of this note). (b) One sentence in Sec. 5 tying the index loop to the path loop. (c) An appendix section with the reader, per-pass and geometry tables. (d) A registered in-batch rank x sign x passes factorial when GPUs return (Sec. 7, E1). The LOOP_RANK contrast should **not** be presented as "a search change at constant capacity": parameters and initialisation are identical, but the function class of the reader is not (Sec. 4).

---

1. Established facts (results files are authoritative)

fact	numbers	status	file
Loop and depth partly recover rank 2 (torus, T = 1024, 900 ep, n = 8, one batch; A and C reused, A s0 reproduced bitwise)	SOLVED (final-5% loss < 0.05) and accuracy: A r2 1 layer 0/8, 0.894; **L r2 + loop x4 2/8, 0.973** (vs A: Fisher 0.47, perm **0.0034**, +0.079 CI [+0.026, +0.131]); **L4 r2 4 layers 5/8, 0.990** (Fisher **0.026**, perm **0.0012**); C r4 8/8, 0.998. L4 - L +0.017 (perm 0.0009). Holm 0.0034 / 0.0028 / 0.0028	registered, verdict **UNMEASURED** (needed L >= 4/8 SOLVED); cutoff sensitivity: at 0.08 L 6/8, L4 8/8	`LOOP_RANK_RESULTS.md`, `LOOP_RANK_ANALYSIS.txt`
Three loss regimes	A 0.05-0.68; L 0.045-0.127; L4 0.011-0.070; C 0.002-0.010: the aids leave A's regime, never enter C's	registered secondary	`LOOP_RANK_RESULTS.md`
L and A identical in parameters and initialisation	204,373 params, max weight difference 0.0 at every seed (gate); L4 799,189	registered gate	`LOOP_RANK_PREREG.md`
Strata (T = 1024)	short / long / wrap-only: A 0.950 / 0.671 / 0.557; L 1.000 / 0.927 / 0.746; L4 1.000 / 0.990 / 0.887; C 1.000 / 0.995 / 0.977	registered secondary	`LOOP_RANK_STRATA.json`, `RANK_MI_STRATA.json`

fact	numbers	status	file
Other lengths	T = 512: A 0.934, L 0.996, L4 0.998, C 1.000; T = 2048: 0.819 / 0.825 / 0.868 / 0.973	registered secondary	`LOOP_RANK_ANALYSIS.txt`
Twice the budget does not rescue plain rank 2	1800 ep: r2 0/8 (0.908) vs r4 7/8 (Fisher 0.0014); 5/8 r2 still descending; the L / L4 arms at 1800 ep were never run (part 2 stopped)	registered	`LOOP_RANK_E1800_P1_RESULTS.md`
Index attention: loop and depth help, do not catch up (torus T = 1024, n = 4, `--fast-attn`, seeds 500-503)	path 1L 1.000; index 1L 0.719, 4L 0.860, loop x4 0.864 (each 0/4, STALLED; complete separations p 0.029). Strata short / long / wrap: 1L 0.769 / 0.480 / 0.482; 4L 0.943 / 0.458 / 0.457; loop 0.938 / 0.507 / 0.509; floors 0.508 / 0.515 / 0.517	registered	`FACTORIAL_RESULTS.md`, `FACTORIAL_*_STRATA.json`
Short-walk torus (T = 128, 300 ep, n = 3)	loop x4 matches 4 real layers at a quarter of the parameters (index horizon 9-16 -> 17-32); on MapFormer L1 0.948 -> loop 0.994 at long intervals	pilot (n = 3)	`LOOPED_PILOT.md`
Match-Query: loop helps path integration	loop vs no loop **+0.346 unpaired** (t 3.75; loop pooled over two batches 0.803 +/- 0.200, 1/16 failures). Per-seed drift between batches 0.185; the paired +0.315 interaction and "never fails" are withdrawn; the pooled contrast is asymmetric (loop 16 runs over 2 batches, no-loop 8 runs over 1); loop matches 3 real layers (+0.032, t 0.30); refining theta per pass adds nothing (+0.090, t 1.56; learned gate	g	<= 0.14)
Match-Query rank x loop (one batch, n = 8)	r2 0.416, r4 0.421, r2 + loop 0.833, r4 + loop 0.986 (8/8 >= 0.941); interaction +0.149	unregistered, one batch	`MQ_RANK_2X2.md`
No per-position router can pay (Match-Query)	loop-count curve flat above k = 3; oracle router +0.007 vs seed sd 0.152	analysis	`LOOP_DEPTH_STRATA.md`

fact	numbers	status	file
Loop beats Level 1.5 under action noise; loop-count sampling makes one pass work	+0.138 / +0.205 at p = 0.10 / 0.25 vs Level15 +0.023 / +0.004 (n = 3, found in a control column); sampled counts {2..6}: count-vs-accuracy spread 0.178 -> 0.001, 0.998 at one pass; the loop's training-length win on the clean torus is convergence (loss-matched +0.006)	unregistered, n = 3-5; needs replication	`.claude-memory/project_loop_and_correction.md`, `L15_LOOP_2X2.md`, `MQ_NOISE_2X2*.md`
Loop x path integration on parity is additive, not super-additive	interaction +0.028 / +0.003 / +0.007 at L = 64 / 128 / 256 (inside MDE); the loop matches or beats 3x its parameters	unregistered	`.claude-memory/project_loop_and_correction.md` (ALGORITHMIC / FRONTIER)

---

2. Design facts that decide how the contrasts can be read

- **The code is computed once.** `MapFormerWM.forward` computes `delta = action_to_lie(token_emb(x))`, `cos, sin = path_integrator(delta)` and hands the same `cos, sin` to every layer. `MapFormerWM_Looped.forward` does the same and applies `layers[0]` `n_loops` times. So A, L and L4 share the code family exactly. L4's three extra blocks never feed theta.
- **L vs A is clean in parameters, initialisation, data and recipe** (registered gate). It is **not** clean in the function class. L computes `B(B(B(B(x))))` with the same weights. It cannot, in general, express A's function (the residual block cannot be switched off for passes 2-4 without switching off pass 1), and it can express functions A cannot (four rounds of attention). "Same parameter count" is not "same capacity" for the reader. It is the same capacity for the code.
- **The loop applies at evaluation as a runtime knob** (`n_loops`), so the per-pass readouts below are exact: the readout after pass j of L *is* L run with `n_loops = j`. For L4, an intermediate readout is a logit lens (out_norm + out_proj on an intermediate residual; the head was trained only on layer 4).
- **Refining theta per pass was tested and declined by the optimiser** (`REFINE_RESULTS.md`; action-noise version in the memory file). Every positive loop result is therefore a result about re-reading a fixed code.

---

3. Theory

T1. A one-pass model reads its code with a fixed, token-typed kernel (THEORY, exact for the code in `model.py`)

Statement. In a one-layer MapWM, the pre-softmax score between the query at position i and the key at position j, head h, is

s_h(i, j) = sum_c |q_c(x_i)| |k_c(x_j)| cos(theta_c(i) - theta_c(j) + psi_c(x_i, x_j)) / sqrt(d_head),

where q_c, k_c are 2-D blocks of W_q LN(e(x_i)), W_k LN(e(x_j)), and psi_c is the angle between them. Both depend only on the token **identities** x_i, x_j. Position enters only through theta(i) - theta(j). The output at i is a function of x_i and of the softmax-weighted mixture of values V(x_j), which also depend on identity alone. So for scored revisit queries (x_i is an action, j ranges over earlier observations), a one-layer model is a **position-only reader of its own code**: a channel-weighted cosine kernel with token-type-dependent weights and phase offsets, a soft vote over the observations it lands on, and an MLP on the vote.

Consequence. A one-layer model's revisit accuracy should be close to that of the best reader of this form fitted to its own theta. More heads and the MLP give a small margin. **Check (ANALYSIS, `phase_reader_soft.py`):** model minus reader overall is +0.028 for A and -0.002 for rank 4 (Sec. 5.3). The check could have failed; it did not.

T2. Extra passes enlarge the reader, not the code (THEORY)

Statement. From pass 2 on, q and k at a position depend on the residual stream, i.e. on context gathered by earlier passes, so both the per-channel weights |q_c||k_c| and the offsets psi_c become context-dependent. The values can carry content retrieved earlier (relay). The code theta is unchanged (Sec. 2). Extra passes can therefore extract from theta, together with context, information that no fixed token-typed kernel can extract. They cannot change which positions theta distinguishes, but they can resolve theta's ambiguities with content.

Predictions. (P1) The code of L and L4 is no better than A's when read by the T1 reader. (P2) L and L4 exceed their own reader by much more than A does. (P3) The gain from extra passes sits in the strata where the code carries information a fixed kernel cannot read. For an index code (no map information) that means short gaps only, where local content matching works; for a partial path code it means long gaps and wrap. (P4) Nothing is gained where one pass already reads a complete code (rank 4, ceiling). **Checks:** P1 holds (reader +0.027 / +0.031 over A, p 0.28 / 0.27; long +0.05 / +0.10, p 0.58 / 0.32; wrap -0.04 / +0.01). P2 holds (Sec. 5.3: p 0.0025 / 0.0045 long, 0.0002 wrap). P3 holds across batches (index loop: short +0.17, long +0.03, wrap +0.03, `FACTORIAL`; rank-2 loop: short +0.05, long +0.26, wrap +0.19, `LOOP_RANK`) and within single models (Sec. 5.7: the index loop's long-gap and wrap-only accuracy sits at 0.50-0.52 after every pass while short gaps climb 0.41 -> 0.94; the rank-2 loop's long gaps climb 0.36 -> 0.93). P4 is untested on the torus (no rank-4 loop arm; MQ_RANK has one on Match-Query: r4 + loop 0.986 vs r4 0.421, so on Match-Query rank 4's one-pass code is not complete).

T3. Why the aids leave A's regime but never enter rank 4's (THEORY with a partial check)

A rank-4 model reads a complete, wrap-consistent 2D code in one pass (reader 1.000 on every stratum), so its loss floor is set by optimisation alone (0.002-0.010). The aids' accuracy is the code's information plus what context can add. Their code does not resolve wrap-only revisits (reader cell top-1 0.04-0.12 on wrap, `phase_code.py`), and a revisit whose unwrapped position is new can be recognised only by content (the local observation

sequence matches one seen in another unwrapped copy of the map). That is unreliable for a reader of any depth. Prediction: the aids' residual loss concentrates in wrap-only and long-gap revisits. **Check (partial):** wrap-only accuracy is 0.747 (L) and 0.887 (L4) against 1.000 on short gaps; the aids' unsolved runs sit at losses 0.05-0.13 with short-gap accuracy 1.000. The loss split by stratum was not computed; see E2.

T4. Search on the code: what the loop does and does not do (THEORY + ANALYSIS + SPECULATION)

The search theory for the rank axis (`RANK_THEORY.md`, `SEARCH_THEORY.md` S6) says that at per-head rank D a clock-free channel cannot also be a full D-dimensional map unless the common mode is removed collectively (R1: z_x + z_-x = 0). The aids' geometry fits this inventory (ANALYSIS, `geometry.py`):

- **A (one pass)**: 7/8 runs end **drifting** (common mode 0.34-2.9 of the map step): the clock-coupled failure.
- **L, L4**: half end drifting (4/8, 3/8), half **cancelling but one-dimensional** (4/8, 4/8): the common mode is gone (0.01-0.05) but the x and y map steps are collinear (sigma_2/sigma_1 0.000-0.002; latent ratio 0.000-0.006). The code is x + kappa y with |kappa| 1.38-1.42 in 4 L runs and 1 L4 run, 0.62-1.72 in the other three.
- **Rank 4**: 8/8 cancelling and two-dimensional (sigma_2/sigma_1 0.72-0.97).

SPECULATION (to be tested, E2). Extra passes make a 1D clock-free code *good enough*. On an unwrapped plane a 1D projection with an irrational slope is injective, but nearly colliding cells exist: for kappa near sqrt 2 the displacements (3, -2), (7, -5), (17, -12) leave phase residuals of only 0.17, 0.07 and 0.03 x-steps. One fixed kernel would have to be very sharp to separate them; a reader that can also check content can reject them. So with a deep reader the collapsed code is a usable basin, while one pass cannot use it (A never lands there: 0/8). Read this way, the aids *change* which basin the step-map search ends in rather than helping it find rank 4's. This is a compensation basin: the reader absorbs the error signal that, at one pass, would keep pushing the code. The registered training curves fit the picture (ANALYSIS, `training_curves.py`). The aids leave the initial plateau early (first epoch below loss 0.5: L 42-67, L4 19-78, against A 183-508 or never, 3/8, and rank 4 83-294) and then creep down without the sharp drop rank 4 shows (rank 4: loss 0.5 -> 0.05 in 57-228 epochs, after a plateau). Not tested: whether the 1D code forms before or after the reader absorbs the short-gap revisits.

What the naive "more gradient paths" account predicts, and why it is not the mechanism (ANALYSIS, `grad_at_init.py`). At initialisation, with weights and batches identical, the 4-pass forward multiplies the step map's gradient norm by 2.9 (2.4-3.8) and the block's by 3.4 (3.0-3.9). Adam is invariant to that scale, and the step-map gradient's batch-to-batch coherence is unchanged (0.46 -> 0.41). The loop does not deliver a stronger or more consistent signal to the code at the start. Its effect on the code's search, if any, runs through the reader (above).

Summary of the theory

code \ reader	one pass	four passes (loop or depth)
none (index RoPE)	short gaps only (0.769 / 0.480 / 0.482)	short gaps better (0.938-0.943); long and wrap still at the floor (FACTORIAL)

code \ reader	one pass	four passes (loop or depth)
rank-2 path code (drifting or 1D)	what a fixed kernel reads (A 0.950 / 0.671 / 0.557; reader 0.940 / 0.558 / 0.448)	context-dependent reading: long 0.93-0.99, wrap 0.75-0.89; code unchanged
rank-4 path code (2D)	complete: 1.000 / 0.995 / 0.977 (reader 1.000)	nothing to add on the torus (untested)

4. Question (2): are loop and depth search aids? How clean is "same parameters, more solved"?

- **As a test of the code's capacity, yes, cleanly.** A, L and L4 have the same code family (Sec. 2). Neither aid changes what the code can represent, and both lift accuracy a long way (registered). The analyses add what the registered numbers could not: the lift is not a better code (P1) but better reading (P2). So the rank-2 deficit is in the code, and more downstream compute or parameters do not repair it. That is a stronger form of "not capacity" than the paper states: the content-to-angle map at rank 2 can represent a solving code (RANK_PROJ), it is not found from scratch, and giving the rest of the network four times the compute and four times the parameters still does not find it (2D code 1/16 vs rank 4 8/8).
- **As "a search change without a capacity change", no.** The pre-registration framed L vs A that way ("the cleanest contrast available"). L has the same parameters and initialisation but a larger reader function class. Its gain is a reader-capacity gain on a fixed code, not search success on the code. The text should say so; the UNMEASURED verdict stands as registered.
- **Strength.** The registered accuracy contrasts are strong (+0.079, p 0.0034; +0.096, p 0.0012; Holm <= 0.0034). The solved counts are knife-edge (L 2/8 at the registered 0.05 cutoff, 6/8 at 0.08; two spot runs at 0.0528 / 0.0538, `REPRO_AUDIT_nonsign.md` note 5). The reader and geometry readings are post hoc, n = 8 per arm, from one batch, and untested against fresh seeds (rule 6).
- **Depth beats the loop** (L4 - L +0.017, p 0.0009; 5/8 vs 2/8). Both have the same code; L4 has four distinct blocks. Under T2 that is a richer reader. The aids' code classes are the same (4/8 1D each).

5. Analyses run (CPU, stored checkpoints; post hoc)

All on the registered LOOP_RANK arms at T = 1024 (A `runs/rank_mi/p0/Vanilla_s*` -> `runs/rank_matched_e900`, L and L4 `runs/loop_rank/p0`, C `runs/rank_mi/p0/Vanilla_r4mi_s*`), seeds 0-7, held-out map (env seed 10000). Evaluation trajectories use np seed 1234 + s, the registered stream; the readers are fitted on np seed 777 + s.

5.1 Reproduction check (`summarize.py`)

The re-implemented forward (returns every pass's logits and attention) matches each model's own forward exactly on the checked trajectory. The trained-readout accuracy over 100 trajectories matches the stored registered evaluations to max |diff| 3.1e-4 (A), 2.8e-4 (L), 9.4e-5 (L4), 3.1e-5 (C).

5.2 Accuracy and attention per pass (`pass_strata.py`, `attn_offsets.py`)

Accuracy by pass (mean of 8 seeds; L: `n_loops = j`, passes 5-8 beyond training; L4: logit lens after layer j):

arm	stratum	1	2	3	4	5	6	7	8
L	all	0.381	0.887	0.964	**0.973**	0.971	0.954	0.899	0.847
L	short	0.382	0.951	0.999	1.000	1.000	0.987	0.932	0.881
L	long	0.363	0.612	0.875	0.927	0.921	0.871	0.799	0.731
L	wrap	0.393	0.519	0.700	0.747	0.730	0.699	0.666	0.623
L4	all	0.507	0.575	0.935	**0.990**				
L4	long	0.500	0.504	0.757	0.990				
L4	wrap	0.504	0.507	0.601	0.887				

The best eval-time pass count is the trained one (4) on 6/8 seeds and 5 on 2/8 (+0.001-0.003). Past it, accuracy falls (8 passes 0.847, two seeds 0.61-0.62). The loop is a fixed-depth computation, not an iteration to a fixed point. This is the free secondary the LOOP_RANK pre-registration listed and never read. One pass of L's trained weights is below the always-blank floor (0.381 vs 0.51): the looped model is not a better one-layer model.

Same-cell attention (scored revisit queries; mass on earlier observation tokens at the same wrapped cell, head 0 / head 1; "argmax" = the most-attended key is a same-cell token):

arm	pass	mass, all	argmax, all	mass, long	mass, wrap	argmax, wrap	entropy (nats)
A	1	0.47 / 0.46	0.69 / 0.70	0.19 / 0.23	0.02 / 0.13	0.04 / 0.19	1.90
L	1	0.03 / 0.01	0.19 / 0.21	0.01 / 0.00	0.00 / 0.00	0.01 / 0.00	6.06
L	2	0.36 / 0.16	0.60 / 0.42	0.12 / 0.04	0.04 / 0.01	0.15 / 0.05	4.13
L	3	0.46 / 0.32	0.75 / 0.66	0.39 / 0.22	0.24 / 0.16	0.36 / 0.27	2.70
L	4	0.52 / 0.38	0.87 / 0.75	0.52 / 0.36	0.32 / 0.20	0.44 / 0.29	2.43
L4	1	0.01 / 0.01	0.04 / 0.09	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	6.46
L4	2	0.10 / 0.15	0.18 / 0.25	0.03 / 0.07	0.00 / 0.03	0.01 / 0.06	4.43
L4	3	0.62 / 0.35	0.80 / 0.56	0.41 / 0.18	0.17 / 0.04	0.25 / 0.04	2.49
L4	4	0.78 / 0.79	0.94 / 0.95	0.67 / 0.73	0.50 / 0.54	0.64 / 0.64	0.71
C	1	0.77 / 0.75	0.92 / 0.94	0.75 / 0.71	0.73 / 0.62	0.88 / 0.80	1.25

Pass 1 of both aids is near-uniform over the prefix (6.1-6.5 nats; uniform over ~1000 keys is ~6.5-6.9). It gathers a summary of the whole context, not the revisit. Retrieval forms over passes 2-4 and sharpens (L4's last layer is sharper than rank 4's single layer, 0.71 vs 1.25 nats). Mass on same-cell **action** tokens, which could hold a relayed observation only after an earlier pass, stays at 0.01-0.12 in every pass: no evidence of a relay through earlier arrivals. On wrap-only revisits L4's last layer finds the right cell 64% of the time, from a code whose own one-pass reader finds it 12% of the time.

5.3 Code quality, read without the block (`phase_code.py`, `phase_reader_soft.py`)

Two readers of each model's own theta, fitted on 24 training-seed walks and scored on the 50 evaluation walks: (i) a nonnegative channel-weighted cosine kernel with top-1 retrieval (`phase_code.py`); (ii) the T1 reader, which adds token-type phase offsets on query and key, a key bias and a soft vote over observations (`phase_reader_soft.py`). (i) is reported as cell retrieval, (ii) as observation accuracy, comparable to the model's.

arm	stratum	model	soft reader (ii)	model - reader	reader (i) cell top-1
A	all / short / long / wrap	0.894 / 0.950 / 0.671 / 0.558	0.866 / 0.940 / 0.558 / 0.448	+0.028 / +0.010 / +0.113 / +0.110	0.688 / 0.800 / 0.182 / 0.105
L	all / short / long / wrap	0.973 / 1.000 / 0.927 / 0.747	0.893 / 0.971 / 0.608 / 0.406	+0.080 / +0.029 / **+0.319** / **+0.341**	0.786 / 0.918 / 0.221 / 0.044
L4	all / short / long / wrap	0.990 / 1.000 / 0.990 / 0.887	0.897 / 0.967 / 0.653 / 0.453	+0.094 / +0.033 / **+0.337** / **+0.435**	0.801 / 0.923 / 0.297 / 0.115
C	all / short / long / wrap	0.998 / 1.000 / 0.995 / 0.977	1.000 / 1.000 / 1.000 / 0.997	-0.002 / 0.000 / -0.005 / -0.020	0.999 / 1.000 / 0.999 / 0.988

Permutation tests (8 vs 8): code quality (ii) L - A +0.027 (p 0.28), L4 - A +0.031 (p 0.27); long +0.050 (0.58) / +0.095 (0.32); wrap -0.042 (0.77) / +0.005 (0.93). C - L +0.107, C - L4 +0.103 (p 0.0002 each; long +0.39 / +0.35, p 0.0002). Model-minus-reader margin vs A's: L overall p 0.0036, long 0.0025, wrap 0.0002; L4 0.0031, 0.0045, 0.0002. Reader (i) gives the same verdicts (code L - A p 0.20, L4 - A p 0.15; C - aids p 0.0002).

Observation-token phase step (the leak, |omega * delta(obs)| / |omega * delta(action)|): A 0.385, L 0.168, L4 0.163, C 0.026. The aids reduce the leak in their 1D-code runs (0.004-0.051) but not in their drifting runs (0.17-0.41).

5.4 Step-map geometry (`geometry.py`)

Per seed: omega-weighted x- and y-step phase vectors (antisymmetric parts of E/W and N/S), dim2 = sigma_2 / sigma_1, cancel = common mode / map (not wrap-aware), kappa, and the stored tail loss. Classes (thresholds post hoc; dim2 is bimodal among cancelling runs, <= 0.002 vs >= 0.57, so the 1D call does not depend on the threshold):

arm	2D map (cancel < 0.1, dim2 > 0.3)	1D map (cancel < 0.1, dim2 <= 0.3)	drifting (cancel >= 0.1)
A	1/8 (s1, the run at 0.998)	0/8	7/8
L	0/8	4/8 (s1, s3, s6, s7; kappa -1.38, -1.40, +1.42, -1.42)	4/8
L4	1/8 (s1, the best L4 run, tail 0.011)	4/8 (s3, s4, s5, s7; kappa -1.60, -0.62, -1.39, -1.72)	3/8
C	8/8	0/8	0/8

Aids pooled vs A, 1D map: 8/16 vs 0/8 (Fisher p 0.022, post hoc). 2D map, aids vs rank 4: 1/16 vs 8/8 (p 1.2e-5). Within the aids, 1D-map and drifting runs reach similar accuracy (L 0.828-0.974 vs 0.872-0.992 on long gaps). Both kinds are compensated by the reader. Per-seed table: `out/GEOMETRY.txt`. Corroboration in the registered delta-space geometry

(LOOP_RANK_GEOMETRY.md): the same four L seeds have opposition 0.03-0.11 and |cos(N, E)| >= 0.9997.

5.5 Training curves (training_curves.py, stored per-epoch losses)

arm	first epoch loss < 0.5	first epoch < 0.15	first epoch < 0.05	shape
A	183-508; never on 3/8	515, 834; never on 6/8	773 (s1); never on 7/8	slow, from a high plateau
L	42-67	261-649	716-870 on 5/8	leaves the plateau at once, then creeps
L4	19-78	198-580	414-899 on 7/8	as L, faster
C	83-294	173-308	261-471	plateau, then a sharp drop (0.5 -> 0.05 in 57-228 epochs)

5.6 Gradient at initialisation (grad_at_init.py)

Same random init per seed, same six batches (B 16, T 1024), same dropout masks, 1 vs 4 passes: step-map gradient norm x2.9 (2.4-3.8), omega x3.0, block x3.4, embedding x1.1, head x1.2. Batch coherence of the step-map gradient 0.46 -> 0.41. No larger relative or more coherent signal reaches the code at the start (Adam removes the scale).

5.7 Index attention per pass (index_loop_passes.py, FACTORIAL checkpoints, seeds 500-503)

Same evaluation stream; trained-depth accuracy matches FACTORIAL_{LOOP,L4}.json to 1e-6 on all 8 models.

model	stratum	pass 1	2	3	4 (trained)	5	6
index loop x4	short	0.408	0.543	0.904	**0.938**	0.917	0.796
index loop x4	long	0.374	0.506	0.509	**0.507**	0.514	0.515
index loop x4	wrap	0.378	0.509	0.510	**0.509**	0.517	0.517
index 4 layers (lens)	short	0.473	0.471	0.620	**0.943**		
index 4 layers (lens)	long	0.510	0.499	0.480	**0.458**		
index 4 layers (lens)	wrap	0.512	0.502	0.476	**0.457**		

Floors (constant per stratum) 0.508 / 0.515 / 0.517. With an index code every pass works on short gaps; long-gap and wrap-only accuracy never leaves the floor (the 4-layer model drifts slightly below it). The rank-2 path loop, on the same readout, moves long gaps 0.363 -> 0.612 -> 0.875 -> 0.927 and wrap 0.393 -> 0.519 -> 0.700 -> 0.747 (Sec. 5.2). That is P3 inside single models: passes add only where the code has something to read.

Not done (and why)

- Match-Query loop checkpoints exist (runs/loop_headroom/PI_loop, PI_flat, PI_L3; runs/mq_rank), but the necessity / destruction runs do not (REPRO_AUDIT_nonsign.md note 8). The reader analysis there needs Match-Query's cell bookkeeping; it is a half-day port (E5).
- Loss by stratum (T3's check): cheap, not run.

---

6. Proposed paper text

6.1 Main text: Sec. 6.1, after "The rank-2 solution exists and is held" (about 190 words)

More computation reads a rank-2 code; it does not repair it. Looping the single block four times (identical parameters and initialisation) or using four real layers raises rank-2 accuracy from 0.894 to 0.973 and 0.990 (permutation p 0.0034 and 0.0012; SOLVED 2/8 and 5/8 against 0/8; rank 4 0.998, 8/8). In MapFormer the phase is computed once from the token embeddings, so neither change touches the code. They change how it is read. A one-layer model reads position through a kernel fixed by token identities. Fitting such a reader to each model's own phase recovers the one-layer models' accuracy (rank 2 0.866 against 0.894; rank 4 1.000 against 0.998), but falls 0.32-0.44 short of the looped and deep models on long-gap and wrap-only revisits. Their codes are no better than plain rank 2's (0.893 / 0.897 against 0.866, p 0.27-0.28) and far from rank 4's (p 0.0002). Run with one pass, the looped weights score 0.381; the right cell is found only at passes 2-4. None of the 16 runs finds rank 4's two-dimensional code: half end with x and y moving the phase along one direction (App. X). Downstream computation compensates for the deficit but does not remove it.

6.2 Main text: Sec. 5, end of "What depth buys index attention" (one sentence)

The same four passes on a rank-2 path code lift long-gap and wrap-only revisits instead (0.671 -> 0.927-0.990 and 0.557 -> 0.747-0.887; Sec. 6.1): passes add accuracy where the code carries position information that one pass cannot read, and none where it carries none.

6.3 Sec. 8 (Discussion), "What the two failure axes share" (one sentence to append)

Giving the rest of the network four times the computation, or four times the parameters, does not find the rank-2 solution either: the deficit is local to the content-to-angle map.

6.4 Appendix X: "Loops and depth on a rank-2 code" (tables from Sec. 5)

- X.1 Registered arms and three loss regimes (table of Sec. 1, rows 1-4).
- X.2 Per-pass accuracy and same-cell attention (Sec. 5.2 tables), with the note that L at `n_loops = j` is exact and L4's intermediate readouts are a logit lens.
- X.3 The one-pass reader: definition (T1), why it bounds a one-layer model up to heads and MLP, and the model vs reader table (Sec. 5.3). State that it is post hoc and fitted on separate walks.
- X.4 Step-map geometry classes (Sec. 5.4).
- X.5 Training curves (Sec. 5.5).
- Caveat paragraph: post hoc; one batch; n = 8; registered verdict UNMEASURED; solved counts sensitive to the cutoff; the reader is an idealised single head, so "the code supports" means "a single fixed-kernel pass can read".

Figure suggestion (`fig_loop_reader`): per stratum (short / long / wrap), bars for A, L, L4, C, each split into "read by one pass" (soft reader) and "added by later passes" (model -

reader). Index 1L / loop from FACTORIAL as a fifth and sixth group with a "cross-batch" label. Data in `out/SUMMARY.txt`; floors from `FACTORIAL_*_STRATA.json` and `RANK_MI_STRATA.json`.

6.5 Limitations (amend the "No recurrent or SSM baselines" bullet)

"Looped (weight-shared) blocks are tested as readers of a fixed code; a loop that recomputes the phase each pass was declined by training on Match-Query and under action noise, and is not tested on the torus."

---

7. Proposed GPU experiments (ranked by value / cost)

Costs from `LOOP_RANK_PREREG.md`: at T = 1024, B 16, one pass ~2.0 s/epoch, loop x4 ~7.4, four layers ~7; 900 epochs: one-pass run ~0.5 h, loop or 4-layer run ~1.8 h, at 2 jobs/GPU.

E1. Registered rank x sign x passes factorial, in one batch (highest value). Torus T = 1024, 900 ep, n = 8, cells {shared r2, shared r4} x {signed, |.|} x {1 pass, loop x4}; 64 runs, 32 x 0.5 h + 32 x 1.8 h = ~74 GPU-h (~19 h on 2 GPUs x 2 jobs). Trimmed: drop r4 |.| (48 runs, ~55 GPU-h). Primary readouts (registered): (a) geometry class per run (2D map / 1D / drifting) and the soft-reader code quality; (b) accuracy. Predictions from T2: the loop does not change the 2D-map count in any cell (r4 signed 8/8 both; r2 <= 2/8 both; |.| <= 2/8 both); the loop raises accuracy in r2 and |.| cells by raising model-minus-reader, not reader; no loop gain in r4 signed (ceiling). Branches: "loop changes the code" if 2D-map counts differ by Fisher $p < 0.05$ in any cell; "reader only" if no count differs and the margin rises. Value: tests the paper's search claim on both axes with one control. A positive "reader only" result is a second axis-general statement; a loop that changes the code would be a direct search-aid result.

E2. Trajectory snapshots of the existing LOOP_RANK arms (medium-high value, cheap if deterministic). Re-run A, L, L4, C for 4 seeds with checkpoints every 50 epochs (A s0 reproduced bitwise, so snapshots should match the stored runs; verify at epoch 900). ~4 x (0.5 + 1.8 + 1.8 + 0.5) = ~18 GPU-h. Readouts on CPU: soft-reader code quality, geometry class and loss by stratum over training. Tests T4's compensation-basin speculation (does the 1D code form after the reader absorbs short-gap revisits?) and T3 (residual loss in long and wrap). Snapshots must not change the RNG stream: check `train_variant.py` before launch (rule 13).

E3. Pass count and code at rank 2 (medium). L at loop x8 and with sampled counts {2..6} (n = 8 each, one batch with L x4): ~2 x 8 x (2.5-3.7 h) = ~50 GPU-h; trimmed to x8 only, ~30 GPU-h. Prediction: more passes raise long-gap and wrap accuracy further and leave the code class unchanged. If wrap stays below rank 4 at any depth, T3 is supported.

E4. Loop vs Level 1.5 under action noise, registered replication (medium value, cheap). The n = 3 finding (+0.205 vs +0.004 at p 0.25, T = 128) fits T2: a noisy code is compensated by a deep reader. n = 8, 3 arms x 2 noise levels at T = 128 (minutes per run): ~4 GPU-h. Readout: accuracy and the soft reader on the noisy code.

E5. Match-Query loop, registered, fresh seeds (medium; fixes a weak citation). The +0.346 is cross-batch and asymmetric, and the necessity runs are gone

(`REPRO_AUDIT_nonsign.md`). One batch: path 1L, path loop x4, path 3L, index 1L, index loop x4, n = 8, fresh seeds, with checkpoints kept: ~5-8 GPU-h. Then port the reader to Match-Query on CPU.

E6. (done on CPU, Sec. 5.7.) Index loop per pass: long and wrap at the floor after every pass.

---

8. What does not survive, and caveats

- "The loop is a search aid at constant capacity" (the H1 framing): constant parameters, larger reader. The registered verdict stays UNMEASURED; the reading in this note is post hoc.
- "More gradient paths": at initialisation the loop scales every gradient about 3x (Adam-invariant) and does not make the step-map gradient more coherent.
- "Iterated attention chains partial matches through earlier arrivals" (relay): mass on same-cell action tokens stays <= 0.12 at every pass; not supported as the main route.
- The reader is an idealised single head with nonnegative channel weights. It is not a proven upper bound on a one-layer model (two heads plus an MLP), but A's margin over it is small (+0.028), so it is close.
- Geometry thresholds are post hoc; "cancel" is not wrap-aware (rule 8). The reader columns are wrap-aware by construction and agree.
- The kappa ~ sqrt 2 clustering (4 of 4 L 1D runs) is an observation, not a claim.
- One batch, n = 8 per arm, no fresh-seed replication of any analysis (rule 6).

9. Reproduce (CPU, from `/home/prashr`)

```
D=mapformer/docs/paper/expansion/scripts/recurrence
for a in A L L4 C; do python3 $D/pass_strata.py $a 100; done      # ~3 min/arm; L runs 8
passes
for a in A L L4 C; do python3 $D/phase_code.py $a; done           # ~30 s/arm
for a in A L L4 C; do python3 $D/phase_reader_soft.py $a; done    # ~4 min/arm
python3 $D/attn_offsets.py; python3 $D/training_curves.py; python3 $D/grad_at_init.py   #
~15 / <1 / ~45 min
python3 -u $D/index_loop_passes.py 100 > $D/out/INDEX_LOOP_PASSES.txt                    #
~3 min at 4 threads
python3 $D/summarize.py > $D/out/SUMMARY.txt; python3 $D/geometry.py > $D/out/GEOMETRY.txt
```